

Simulation Execution Documentation Module (SEDM)

OOF™ Origin Open Foundation™

Independent Methodological Authority

Architecture Ecosystem: Structured Reality Standards™

Architecture Family: SIMULOS® — Simulation Governance Architecture

Parent Standard: Simulation Execution Standard (SES)

Operational Layer: Simulation Execution Governance Layer

Category: Governance & Enforcement

Subcategory: Simulation Governance

Type: Parent Standard Module

Governed Space: Simulation Execution Documentation

Version: 1.0

Status: Canonical · Open Module

Effective Date: 19 August 2026

Compatibility: OOF® Methodology OS™ · GOA™ · ORA™ · VALIDOS® · INTEGROS® · ADIT® · AGA™ · AIG® · ASGA™ · RIS™

AI-Readable: Yes

Authority: OOF®

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Governed Space

Simulation Execution Documentation

Canonical Definition

Simulation Execution Documentation Module (SEDM) defines the governance framework through which materially relevant information about Simulation Execution is captured, structured, linked, preserved, version-bounded, and made sufficiently reconstructable to establish what was actually executed, under which configuration and conditions, what occurred during execution, which interventions and deviations arose, how repetitions were related, whether execution was completed, and which Simulation Outputs originated from that execution.

SEDM converts distributed execution information into a governed:

Simulation Execution Record.

Its purpose is not to require documentation for documentation's sake.

Its purpose is to ensure that material Simulation Outputs and downstream claims do not become detached from the execution history required to understand how they were produced.

Operational Role

SEDM serves as the fifth and final internal governance space of the Simulation Execution Standard (SES).

The preceding modules establish:

SIM — What exact simulation instance was instantiated?

SECM — How was active execution controlled?

SOM — What materially relevant events and behaviors occurred during execution?

SRPM — How were executions repeated, related, varied, and compared?

SEDM asks:

Can the complete materially relevant execution history now be reconstructed sufficiently to determine what actually ran, what occurred, how the execution changed, and which outputs it produced?

The governed progression is:

Simulation Instantiation Record

↓

Execution Control Record

↓

Simulation Observation Record

↓

Simulation Repetition Record

↓

Execution Documentation Assembly

↓

Execution Traceability

↓

Execution Provenance

↓

Execution Lineage



Execution Completeness Documentation



Execution-to-Output Linkage



Governed Simulation Execution Record

Module Operational Space

SEDM governs: execution documentation requirements, Execution Record identity, documentation scope, documentation granularity, documentation timing, execution metadata, configuration documentation, model-version documentation, environment-version documentation, scenario documentation, input documentation, parameter documentation, initialization documentation, authorization documentation, execution-state documentation, runtime-condition documentation, control-action documentation, intervention documentation, deviation documentation, observation documentation, anomaly documentation, interruption documentation, restart documentation, termination documentation, failure documentation, completion documentation, repetition documentation, randomization documentation, seed documentation where applicable, execution provenance, execution lineage, campaign documentation, run-to-campaign relationships, documentation completeness, documentation gaps, documentation uncertainty, execution reconstruction, point-in-time reconstruction, execution-to-output traceability, record retention, record continuity, and formation of the Governed Simulation Execution Record.

Module Function

SEDM exists to prevent: simulation outputs with no identifiable originating execution, intended configurations being substituted for actual executed configurations, model versions becoming detached from execution records, undocumented runtime interventions, execution deviations disappearing after completion, failed and interrupted runs being removed from history, observations losing their execution context, repetitions becoming indistinguishable, seeds or stochastic conditions becoming untraceable, simulation campaigns becoming collections of anonymous output files, later reconstruction depending upon individual memory, partial documentation being represented as complete documentation, and downstream validation or audit being forced to infer what happened during execution.

Simulation Execution Record

A Simulation Execution Record is the governed documentation structure representing a specific Simulation Execution and its materially relevant execution history.

Conceptually:

Execution Identity Instantiated Configuration Execution Conditions Execution

Controls Observed Events Interventions Deviations Execution State History
Repetition Context Completion Status Output Linkage

=

Governed Simulation Execution Record

Execution Record Identity

Every materially relevant Execution Record should be bound to an identifiable Simulation Execution.

Conceptually:

Simulation Execution ID → Simulation Execution Record

The record should not exist merely as an anonymous log file or output directory.

Documentation Scope

Documentation requirements should correspond to the Simulation Purpose, consequence, complexity, execution environment, and downstream reliance.

SEDM does not require every technical event to be permanently recorded.

It requires:

materially relevant execution information to remain sufficiently available for the intended governance purpose.

Documentation Granularity

Documentation may operate at different levels:

Run Level

Episode Level

Batch Level

Scenario Level

Campaign Level

or another defined execution unit.

Large-scale simulation may combine these levels.

Run-Level Documentation

Run-level documentation may preserve: Execution ID, configuration, inputs, parameters, seed, execution state, interventions, deviations, observations,

completion status, and outputs.

Campaign-Level Documentation

Campaign-level documentation may preserve shared conditions across large populations of executions.

For example:

Campaign C-001

may define: model version, environment version, common scenario family, infrastructure, orchestration configuration, repetition rules, and shared execution controls.

Individual runs then document material differences from that common basis.

No Duplicate Documentation Requirement

SEDM does not require identical information to be copied into every run record where a governed shared Campaign Record already establishes that information.

The requirement is:

traceable inheritance.

Documentation Inheritance

A run may inherit documented conditions from a Campaign Record where: the inherited condition is explicitly linked, the relevant version is stable, exceptions remain visible, and the run can be reconstructed without ambiguity.

No Assumed Inheritance Principle

SIMULOS® establishes:

A run must not be assumed to inherit campaign conditions where material evidence shows that its actual execution differed.

Actual execution takes precedence over intended campaign configuration.

Actual Configuration Documentation

SEDM should preserve the materially relevant configuration that actually entered execution.

This may include: Simulation Object version, model version, environment version, scenario version, Input Set, parameter set, software build, hardware configuration, dependencies, randomization basis, and initialization state.

Intended vs Executed Configuration

SEDM distinguishes:

Intended Configuration

from:

Executed Configuration.

Where these differ, the difference becomes part of the Execution Record.

Configuration Delta Record

A Configuration Delta Record may identify:

Expected Configuration → Actual Configuration → Material Difference

This supports later interpretation of unexpected behavior.

Execution State History

SEDM may preserve state transitions such as:

Prepared

↓

Initialized

↓

Running

↓

Paused

↓

Restarted

↓

Running

↓

Completed

This is materially different from a record that merely states:

Completed.

Execution Timeline

Where timing is material, SEDM may preserve an Execution Timeline connecting:

Initialization

Start

Control Events

Observations

Interventions

Anomalies

Interruptions

Restarts

Termination

and:

Completion.

Event Ordering

The order of execution events may itself be material.

SEDM should therefore preserve event ordering where causal interpretation depends upon sequence.

Timestamp Governance

Where timestamps are used, the record should preserve sufficient context to determine what they represent.

This may include: simulation time, wall-clock time, mission time, relative execution time, or another defined temporal reference.

Simulation Time ≠ Wall-Clock Time

A simulation may represent:

24 hours of operational behavior

while executing in:

12 minutes of wall-clock time.

SEDM preserves which temporal frame an event belongs to.

Execution Control Documentation

Material execution-control actions should remain linked to the execution.

Examples include: pause, resume, termination, resource restriction,

synchronization, scenario progression, injected event, controlled failure, recovery command, or another runtime control.

Intervention Documentation

A material intervention should document where applicable:

Who or what intervened

When

Under which authority

What changed

Why

and:

Which part of execution may have been affected.

Human Intervention

Where humans intervene during execution, SEDM may preserve: role, intervention type, decision point, timing, authority, and resulting execution effect.

AI or Autonomous Intervention

Where an AI or autonomous system modifies simulation execution, SEDM may preserve: initiating system, decision basis where available, authorized capability, action performed, affected parameters or scenario state, and resulting execution path.

No Invisible Intervention Principle

A material intervention capable of changing Simulation Execution must not disappear from the execution history.

Deviation Documentation

A material Execution Deviation should remain connected to: the expected condition, actual condition, time or execution stage, affected execution component, response, and resulting execution status.

Deviation Resolution

A deviation may be:

Resolved During Execution

Accepted

Conditionally Accepted

Unresolved

Execution-Limiting

or:

Execution-Terminating.

SEDM documents the status.

It does not independently determine downstream validity.

Observation Documentation

Material observations produced under SOM should remain linked to the execution context in which they occurred.

An observation should not become:

Behavior X occurred

without sufficient ability to determine:

during which execution, scenario, state, configuration, and conditions Behavior X occurred.

Observation Context Preservation

Where material:

Observation → Execution ID → Execution State → Scenario → Configuration → Time Context

should remain traceable.

Observation Limitation Documentation

If portions of execution were not observable, that limitation should remain part of the Execution Record.

Examples include: telemetry loss, missing internal state, insufficient sampling, sensor dropout, logging failure, or inaccessible agent state.

No Full-Observation Assumption

SIMULOS® establishes:

Absence of a recorded event does not establish absence of the event where the relevant execution region was not sufficiently observable.

Anomaly Documentation

Material anomalies should remain visible even where the execution ultimately completes successfully.

Anomaly records may include: anomaly identity, observation time, affected component, execution context, response, persistence, and relationship to resulting outputs.

Failure Documentation

Failed Simulation Executions remain part of the governed execution history where material.

A failure record may preserve: failure state, failure point, observed cause where known, affected scenario, affected configuration, recovery attempt, and output status.

Failure Cause ≠ Failure Observation

SEDM distinguishes:

Observed Failure

from:

Determined Cause of Failure.

The execution record should not present an unverified causal explanation as established fact.

Interrupted Execution

An interrupted execution should remain distinguishable from:

Completed Execution

and:

Failed Execution.

An interruption may later be restarted or abandoned.

Restart Documentation

Where execution restarts, SEDM should preserve whether the restart: resumed from prior state, restored from checkpoint, restarted from initialization, changed configuration, changed input, changed seed, or created a new Execution ID.

Execution Continuity

A restarted execution should not automatically be treated as continuous with the original run.

The continuity relationship should be explicitly governed.

Termination Documentation

Termination may be:

Normal

Rule-Based

Human-Initiated

AI-Initiated

Safety-Initiated

Failure-Induced

or another defined condition.

Where material, the termination basis should remain visible.

Execution Completion Documentation

SEDM preserves the Execution Completeness status established under SES.

Possible statuses may include:

Complete

Conditionally Complete

Partially Complete

Incomplete

Failed

or:

Undetermined. Completion ≠ Success

SIMULOS® establishes:

A Simulation Execution may be completely executed while the simulated system fails catastrophically.

Execution completion describes execution status.

It does not describe desirability of the simulated outcome.

Repetition Documentation

SEDM preserves the repetition structure established by SRPM.

This may include: Repetition Set ID, Reference Execution, controlled constants, permitted variation, seed relationship, execution order, run dependency, repetition drift, comparability status, and repetition sufficiency.

Repetition Lineage

A repetition history may be represented:

Run R001

↓

Unexpected Divergence

↓

Focused Repeat R002

↓

Focused Repeat R003

↓

Rare Behavior Reproduced

↓

New Scenario Requirement

SEDM preserves this lineage so that the discovery pathway remains reconstructable.

Randomization Documentation

Where randomness materially affects execution, SEDM may preserve: randomization method, random seed, sampling configuration, stochastic parameter state, and relationship between randomization and execution.

No Seed-Only Reconstruction Principle

A seed should not be treated as a complete execution record.

Reconstruction may also depend upon: model version, environment version, dependencies, initialization, inputs, hardware, software, timing, and execution controls.

Execution Provenance

Execution Provenance establishes the origin of the Simulation Execution Record and its material components.

It may connect:

Execution Record

to: simulation system, execution engine, orchestration platform, infrastructure, configuration source, input source, scenario source, observation source, and responsible execution authority where applicable.

Provenance ≠ Integrity

SEDM governs execution-specific provenance relationships.

General integrity properties remain governed under INTEGROS®.

Execution Lineage

Execution Lineage establishes relationships between Simulation Executions over time.

Examples include:

Baseline → Repeat

Baseline → Modified Parameter Run

Failure → Recovery Run

Scenario → Stress Escalation

Model v1 Run → Model v2 Comparative Run

Initial Campaign → Follow-Up Campaign Why Lineage Matters

Without lineage, a simulation repository may contain millions of outputs while losing the reasoning path that explains:

why later executions were performed.

SEDM preserves that relationship.

Execution Reconstruction

Execution Reconstruction is the ability to recover the materially relevant execution basis after the original execution has ended.

A reconstruction should answer, where applicable:

What was executed?

Which versions were used?

Which scenario ran?

Which inputs entered?

Which parameters were active?

Which randomization conditions applied?

What occurred during execution?

Which interventions occurred?

Which deviations occurred?

Was the execution interrupted or restarted?

Was it part of a Repetition Set?

Was execution complete?

Which outputs came from it? Reconstruction ≠ Reproduction

A simulation may be sufficiently documented for reconstruction even where exact technical reproduction is impossible.

For example, external dependencies may no longer exist.

SEDM distinguishes:

Can we determine what happened?

from:

Can we execute exactly the same run again?

Point-in-Time Reconstruction

SEDM should support reconstruction of the execution state at a materially relevant point in time.

For example:

What was the active parameter configuration when the anomaly occurred?

or:

Had the human intervention already occurred when the autonomous recovery behavior began?

Causal Sequence Preservation

SEDM does not determine causality.

However, it should preserve sufficient temporal and execution relationships to

allow downstream analysis to investigate causality.

Documentation Completeness

Documentation Completeness asks:

Does the Execution Record contain the materially required information necessary for its intended governance and downstream use?

Possible statuses may include:

Complete

Conditionally Complete

Partially Complete

Incomplete

or:

Completeness Undetermined. Documentation Completeness $\neq$ Execution Completeness

These are separate conditions.

A simulation may be:

Execution Complete + Documentation Incomplete

or:

Execution Failed + Documentation Complete.

This distinction is critical.

Documentation Gap

A Documentation Gap exists where materially relevant execution information is missing, inaccessible, ambiguous, or insufficiently linked.

Examples include: unknown model version, missing seed, unknown intervention, missing runtime log, unidentified scenario revision, missing input version, unknown restart state, or orphan output.

Documentation Gap Classification

A gap may be:

Minor

Material

Critical

or:

Undetermined

relative to the intended downstream use.

No Documentation Inflation Principle

SIMULOS® establishes:

A partially documented execution must not be represented as fully reconstructable merely because large volumes of technical logs exist.

Volume of logging is not equivalent to documentation sufficiency. Machine Logs
≠ Governed Documentation

Raw logs may be important evidence.

But a collection of logs does not automatically establish: execution identity, configuration binding, lineage, material event classification, completion status, or output linkage.

SEDM provides the governance structure around technical records.

Documentation Uncertainty

Where documentation cannot establish a material fact with sufficient confidence, the uncertainty should remain visible.

Possible representation:

Known

Conditionally Known

Inferred

Unknown

or another governed classification.

No Retrospective Certainty Principle

SIMULOS® establishes:

Missing execution information must not later be converted into certainty merely because a downstream conclusion requires it.

Execution-to-Output Traceability

Every material Simulation Output should remain traceable to the Simulation Execution or governed execution population that produced it.

Conceptually:

Simulation Execution ID

↓

Output ID

or:

Simulation Campaign / Repetition Set

↓

Aggregated Output ID Output Origin

SEDM establishes the execution-side provenance of an output.

SOGS then governs:

what the output is, how it is classified, how it is bounded, and what may legitimately be inferred from it.

No Orphan Output Principle

A material Simulation Output should not become detached from the execution basis that produced it.

Aggregated Output Traceability

An aggregated result may represent: multiple runs, a Repetition Set, a Scenario Set, or a complete Simulation Campaign.

SEDM should preserve the population of executions contributing to the aggregate where material.

No Anonymous Aggregation Principle

SIMULOS® establishes:

An aggregate Simulation Output should not conceal which materially relevant executions were included, excluded, failed, or unavailable.

Excluded Execution Documentation

Where runs are excluded from aggregation or analysis, the exclusion may itself require documentation.

This can prevent: cherry-picking, survivor bias, silent removal of failed runs, and post hoc population manipulation.

Campaign Documentation

Large Simulation Campaigns may require hierarchical documentation:

Campaign Record



Scenario Families



Repetition Sets



Individual Executions



Outputs

This structure allows governance at scale without losing material run-level traceability.

Large-Scale Execution Documentation

A campaign containing millions or billions of runs cannot always preserve every technical artifact indefinitely.

SEDM therefore supports risk- and purpose-appropriate documentation strategies while protecting material reconstructability.

Possible approaches include: structured metadata, hierarchical records, exception-focused retention, representative execution preservation, aggregate lineage, checkpoint preservation, reproducibility packages, and selective high-resolution logging.

Selective Retention

Selective retention may be legitimate where the selection logic itself is governed.

For example, an organization may preserve: all failed runs, all anomalous runs, all rare-event runs, representative successful runs, complete metadata for every run, and aggregated technical logs.

No Selective Success Preservation

Principle

Retention policy must not systematically preserve favorable executions while eliminating materially relevant unfavorable executions.

Record Retention

Retention periods may depend upon: simulation purpose, consequence, regulatory context, contractual requirements, validation use, audit requirements,

operational reliance, incident exposure, and reproducibility needs.

SEDM does not prescribe one universal retention period.

Documentation Continuity

Documentation should remain sufficiently interpretable across: software changes, model changes, environment changes, personnel changes, organizational changes, storage migrations, and simulation-platform replacement.

Format Obsolescence

Long-lived simulation programs may lose access to historical records because: proprietary formats disappear, software versions become unavailable, dependencies expire, or storage systems change.

SEDM treats continued interpretability as part of documentation continuity where materially required.

Documentation Versioning

Execution documentation structures may evolve.

Where schema changes affect interpretation, the applicable documentation version should remain identifiable.

AI-Readable Execution Records

Where technically appropriate, SEDM supports machine-readable records capable of connecting:

Object → Model → Environment → Scenario → Input → Execution → Observation → Repetition → Output

This can support automated: simulation governance, anomaly investigation, validation, audit, comparison, and future simulation planning.

Automated Documentation

Simulation platforms may automatically generate substantial portions of the Execution Record.

Automation is encouraged where appropriate.

However:

Automated capture ≠ automatically sufficient governance.

The record still needs to capture the materially relevant relationships required by SEDM.

AI-Generated Documentation

AI systems may summarize execution histories, classify anomalies, or construct documentation records.

Where AI-generated documentation introduces interpretation rather than direct observation, that distinction should remain visible.

Observed Fact vs Generated Summary

SEDM should preserve the difference between:

Machine-Captured Execution Event

and:

AI-Generated Description of the Event.

This prevents generated summaries from silently replacing source execution evidence.

Autonomous Simulation Campaigns

Future autonomous systems may: generate scenarios, launch simulations, analyze observations, select follow-up runs, terminate low-value branches, and generate new campaigns

without continuous human direction.

SEDM becomes particularly important because the execution pathway itself may be machine-generated.

Autonomous Execution Lineage

For autonomous campaigns, documentation should preserve where material:

Trigger

↓

Simulation Selection

↓

Execution

↓

Observation

↓

Machine Decision

↓

Follow-Up Execution



Result

This allows later reconstruction of why the simulation system chose to execute what it did.

No Hidden Autonomous Path Principle

When an autonomous system materially determines the simulation path, that path should remain reconstructable rather than appearing as an unexplained collection of simulation outputs.

Space and Mission Simulation

Space agencies and aerospace organizations may run enormous simulation campaigns before: launch, docking, landing, autonomous navigation, recovery, fault response, or mission-critical maneuvers.

The challenge may not be generating simulation data.

The challenge may be answering months later:

Which exact spacecraft configuration produced this anomaly?

Which environmental assumptions applied?

Which run first revealed the failure mode?

Was the anomaly reproduced?

Which software version was running?

Was that run included in the final aggregate?

Which follow-up simulations were triggered by it?

SEDM provides the documentation structure required to preserve these relationships.

Autonomous Vehicle Simulation

An autonomous-vehicle organization may execute hundreds of millions of simulated driving episodes.

A later incident may require comparison with historical simulation.

SEDM allows the organization to determine: whether the relevant condition had been simulated, which execution population contained it, what system version was tested, what behavior occurred, whether anomalous runs existed, and whether those runs were included in downstream analysis.

Simulation Documentation Bottleneck

At simulation scale, the bottleneck may shift from:

Can we run enough simulations?

to:

Can we reconstruct why the simulations produced what they produced?

Organizations may possess enormous computational capacity while lacking governed relationships between: execution, configuration, observation, repetition, anomaly, and output.

SEDM defines that relationship as a governable space.

Relationship to SIM

SIM — Simulation Instantiation Module establishes the exact executable simulation instance.

SEDM preserves the resulting instantiation record as part of the permanent Execution Record.

Relationship to SECM

SECM — Simulation Execution Control Module governs runtime execution control.

SEDM preserves materially relevant control events, interventions, deviations, and execution-state transitions.

Relationship to SOM

SOM — Simulation Observation Module governs observation of materially relevant execution behavior.

SEDM preserves those observations with their execution context.

Relationship to SRPM

SRPM — Simulation Repetition Module governs repeated executions.

SEDM preserves Repetition Sets, run relationships, variation, drift, comparability, and repetition lineage.

Relationship to SOGS

SEDM provides the formal handoff from execution to output governance.

The relationship is:

Governed Simulation Execution Record

↓

Simulation Output



SOGS — Simulation Output Governance Standard

SEDM establishes:

Where did this output come from?

SOGS establishes:

What exactly does this output represent, and what may legitimately be inferred from it?

Relationship to ADIT®

ADIT® may audit evidence derived from Simulation Execution.

SEDM ensures that the execution-side record exists and remains sufficiently structured for audit examination.

The boundary remains:

SIMULOS® governs Simulation Execution Documentation.

ADIT® governs audit activity over relevant evidence and records.

Relationship to INTEGROS®

INTEGROS® governs general integrity properties.

SEDM does not duplicate general integrity governance.

It governs the simulation-specific structure, context, lineage, and reconstructability of execution documentation.

Relationship to VALIDOS®

VALIDOS® may use Simulation Execution Records and Simulation Outputs as part of a validation basis.

SEDM does not determine whether those records are sufficient validation evidence.

It ensures that the simulation execution from which such evidence originates remains governable.

Relationship to ORA™

ORA™ governs Operational Reality.

SEDM documents what occurred inside the Simulation Execution.

A complete Simulation Execution Record does not establish that simulated

behavior corresponds to Operational Reality.

Governance Boundary

SEDM begins when materially relevant records from Simulation Instantiation, Execution Control, Observation, and Repetition require consolidation into a persistent and reconstructable execution history.

SEDM ends when the Simulation Execution has been sufficiently: identified, configuration-bound, temporally documented, state-documented, intervention-documented, deviation-documented, observation-linked, repetition-linked, provenance-linked, lineage-linked, completeness-assessed, and connected to its resulting Simulation Outputs

to form a Governed Simulation Execution Record.

SEDM does not determine: output meaning, output classification, legitimate inference, simulation validity, correspondence with Operational Reality, validation outcome, or operational reliance.

Minimum Implementation Framework

1. Establish Execution Record Identity

Bind documentation to the relevant Simulation Execution ID or Campaign ID. 2. Capture the Executed Configuration

Preserve the materially relevant actual: object version, model version, environment version, scenario, inputs, parameters, dependencies, initialization, and randomization basis.

3. Preserve Execution State History

Document material transitions from initialization through completion, interruption, failure, or termination. 4. Document Material Controls and Interventions

Preserve actions capable of materially affecting execution. 5. Document Deviations and Anomalies

Keep materially relevant departures from intended execution visible. 6. Link Observations to Execution Context

Preserve where and under which conditions material observations occurred. 7. Preserve Repetition Relationships

Connect repeated runs to their Reference Execution and Repetition Set. 8. Establish Execution Provenance and Lineage

Record where the execution originated and how it relates to prior and subsequent executions. 9. Assess Documentation Completeness

Identify material Documentation Gaps and uncertainty. 10. Establish Execution-to-Output Traceability

Bind material Simulation Outputs to their originating execution or execution

population. 11. Preserve Documentation Continuity

Ensure records remain interpretable for the required lifecycle period. 12.
Establish the Governed Simulation Execution Record

Consolidate the material execution history into the formal downstream handoff to SOGS.

Governance Outputs

SEDM may produce: Simulation Execution Record, Simulation Campaign Record, Execution Documentation Requirement, Execution Metadata Record, Executed Configuration Record, Configuration Delta Record, Model-Version Execution Record, Environment-Version Execution Record, Scenario Execution Record, Input Execution Record, Parameter Execution Record, Initialization Record, Execution Authorization Record, Execution State History, Execution Timeline, Temporal Reference Record, Execution Control Documentation, Intervention Record, Human Intervention Record, Autonomous Intervention Record, Execution Deviation Record, Deviation Resolution Record, Observation Context Record, Observation Limitation Record, Anomaly Record, Failure Record, Interruption Record, Restart Record, Termination Record, Execution Completion Record, Repetition Documentation Record, Repetition Lineage Record, Randomization Record, Seed Record, Execution Provenance Record, Execution Lineage Map, Execution Reconstruction Record, Point-in-Time Reconstruction Record, Documentation Completeness Assessment, Documentation Gap Record, Documentation Uncertainty Record, Execution-to-Output Traceability Record, Aggregated Output Lineage Record, Excluded Execution Record, Documentation Retention Record, Documentation Continuity Record, and Governed Simulation Execution Record.

Use Case 1 — Autonomous Spacecraft Mission Simulation

Scenario

A space organization runs a large pre-mission simulation campaign for autonomous spacecraft recovery.

Several months later, engineers discover an unusual recovery behavior and need to determine whether it had appeared during simulation. Application

SEDM connects the relevant historical execution to: spacecraft software version, simulation model version, environment version, scenario, navigation inputs, power state, random seed, execution observations, runtime interventions, and follow-up repetitions.

The original anomaly can be reconstructed and its lineage traced into later simulation runs. Result

Instead of searching through disconnected logs and output files, the organization can establish:

which exact execution first produced the behavior, under which conditions, whether it was repeated, and which downstream outputs incorporated it. Use Case 2 — Autonomous Vehicle Simulation Campaign

Scenario

An autonomous-driving developer executes millions of simulated driving episodes across changing software versions.

A rare operational incident later raises the question of whether a similar condition had previously appeared in simulation. Application

SEDM allows historical Simulation Execution Records to be queried by: scenario conditions, vehicle software version, environment version, observed behavior, anomaly classification, execution completion, and output lineage.

The organization discovers that similar behavior occurred in several historical runs, but some of those executions had been excluded from an aggregate because they were classified as incomplete. Result

SEDM makes the exclusion and its execution context visible.

The organization can determine what actually occurred without retroactively treating the aggregate output as a complete representation of all historical executions.

Architectural Position

Simulation Execution Documentation Module (SEDM) is the fifth and final internal governance space of the Simulation Execution Standard (SES).

The complete SES progression is:

SIM — Simulation Instantiation

↓

SECM — Simulation Execution Control

↓

SOM — Simulation Observation

↓

SRPM — Simulation Repetition

↓

SEDM — Simulation Execution Documentation

↓

Governed Simulation Execution Record

This closes the Simulation Execution governance space.

The resulting record becomes the governed handoff into:

9. Simulation Output Governance Standard (SOGS).

Foundational Principle

Simulation Execution becomes governable beyond the moment of execution only when the material configuration, conditions, events, interventions, deviations, repetitions, provenance, lineage, and resulting outputs remain connected through a reconstructable execution record.

Canonical Closing Statement

Simulation Execution Documentation Module completes the Simulation Execution Standard by transforming materially relevant execution information into a persistent, structured, and reconstructable Governed Simulation Execution Record. Through execution identity, actual-configuration documentation, state and timeline preservation, intervention and deviation records, observation context, failure and restart documentation, repetition lineage, provenance, documentation-completeness assessment, campaign-scale traceability, and execution-to-output linkage, SEDM prevents Simulation Outputs from becoming detached from the computational events that produced them. It establishes the formal execution record through which individual runs, repeated executions, large-scale campaigns, autonomous simulation pathways, anomalies, failures, exclusions, and resulting outputs can remain traceable across the simulation lifecycle and provides the governed handoff from Simulation Execution into Simulation Output Governance.

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>